Inteligência Artificial no Apoio à Programação Competitiva: Fundamentos, Arquitetura e Avaliação Exploratória do AutoTutor

Nome do autor[1]

[1]Instituição e departamento a preencher
Cidade – Estado – Brasil

email@instituicao.br

Abstract. *This paper presents the AI foundations of AutoTutor, a competitive programming learning platform combining language models, retrieval-augmented generation, learner evidence, and executable verification. It distinguishes model training from inference and external memory, and explains lexical retrieval, embeddings, and personalization. The prototype offers 39 local task contracts and 302 external texts. Exploratory experiments compare three retrieval strategies over 12 queries and examine five responses from a lightweight local model. Observed retrieval and reasoning failures motivate explicit provenance and independent validation. The results characterize implementation limits; they do not establish educational effectiveness or general model superiority.*

Resumo. *Este artigo apresenta os fundamentos de IA do AutoTutor, plataforma de aprendizagem de programação competitiva que integra modelos de linguagem, geração aumentada por recuperação, evidências do aluno e verificação executável. Distinguem-se treinamento, inferência e memória externa, explicando recuperação lexical, embeddings e personalização. O protótipo oferece 39 contratos locais e 302 textos externos. Experimentos exploratórios comparam três estratégias de recuperação em 12 consultas e examinam cinco respostas de um modelo local leve. As falhas observadas motivam proveniência explícita e validação independente. Os resultados caracterizam limitações da implementação, sem demonstrar eficácia educacional ou superioridade geral de modelos.*

1. Introdução

Aprender programação competitiva exige reconhecer padrões, relacionar estruturas de dados a operações, justificar algoritmos e interpretar erros de execução. Encontrar material sobre um assunto não basta: o estudante também precisa transformar a leitura em prática e receber retorno sobre suas tentativas. Modelos de linguagem podem apoiar esse percurso por meio de explicações, exemplos e propostas de exercícios. Entretanto, uma resposta bem escrita pode conter uma fórmula incorreta, um exemplo contraditório ou uma referência que não foi consultada.

O problema investigado neste trabalho é como organizar uma aplicação educacional que utilize IA sem confundir geração plausível com conhecimento verificado. O

AutoTutor foi desenvolvido como protótipo funcional, em Python e Streamlit, com persistência local e juiz C++17. A questão de pesquisa orientadora é: *como integrar recuperação de conhecimento, personalização e geração de problemas com mecanismos que tornem suas limitações observáveis?*

A contribuição está na integração e análise de decisões de engenharia: separar texto de referência de metadados; recuperar conteúdo a partir do pedido específico; fornecer evidências resumidas do aluno; limitar a geração de problemas a contratos verificáveis; e avaliar separadamente recuperação, geração e execução. Não se propõe um novo modelo fundacional, nem se relata treinamento de pesos ou experimento controlado com estudantes. O foco é explicar os conceitos de IA utilizados, documentar sua implementação e discutir evidências técnicas preliminares.

2. Fundamentação teórica de inteligência artificial

2.1. Modelo, treinamento, inferência e conhecimento paramétrico

Um modelo de linguagem representa uma distribuição condicional sobre sequências de tokens. Dado um contexto x e uma saída $y = (y_1, \ldots, y_T)$, a geração autorregressiva pode ser descrita por

$$p_\theta(y \mid x) = \prod_{t=1}^{T} p_\theta(y_t \mid x, y_{<t}), \tag{1}$$

na qual θ representa os parâmetros aprendidos. Treinar corresponde a ajustar esses parâmetros a partir de dados e de uma função de perda. Inferir corresponde a usar parâmetros já existentes para produzir uma saída. Um pedido enviado por API é uma operação de inferência; salvar respostas no SQLite também não altera, por si só, os parâmetros do modelo.

No AutoTutor, o conhecimento paramétrico é aquele mobilizado pelo modelo escolhido antes de receber os textos recuperados da biblioteca. Ele pode conter informações úteis sobre algoritmos, mas sua origem não é necessariamente identificável em cada resposta. O projeto não presume que o modelo esteja "aprendendo continuamente" com todos os materiais salvos. A memória persistente permite recuperar evidências em solicitações posteriores, o que é diferente de *fine-tuning*. Essa distinção evita atribuir à aplicação um processo de treinamento que não foi implementado.

O ajuste fino seria uma etapa adicional, com conjunto de treinamento, objetivo de otimização, controle de versões e avaliação em dados separados. Além do custo computacional, exemplos incorretos gerados pelo próprio sistema poderiam reforçar erros se fossem reutilizados sem revisão. Por isso, a arquitetura atual prioriza memória externa e evidências rastreáveis, mantendo os pesos do modelo inalterados.

2.2. Transformers, atenção e contexto

O Transformer utiliza atenção para relacionar posições de uma sequência [Vaswani et al. 2017]. Na atenção por produto escalar escalonado, consultas, chaves e valores são representados por matrizes Q, K e V:

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V. \tag{2}$$

A ponderação resultante permite combinar informações de posições relevantes. Em geração causal, posições futuras são mascaradas; o próximo token depende apenas do contexto disponível. Múltiplas cabeças permitem aprender diferentes relações entre representações.

Na aplicação, isso significa que o pedido, os exemplos e os trechos recuperados disputam uma janela finita de contexto. Enviar toda a biblioteca não equivale a tornar a resposta melhor: aumenta custo, pode truncar o pedido e introduz informação irrelevante. O sistema delimita o tamanho dos trechos enviados e preserva a prioridade do objetivo atual. A arquitetura interna de cada provedor não é implementada pelo AutoTutor; ela é acessada por uma camada de inferência.

2.3. Decodificação, tamanho do modelo e recursos

Na decodificação, o modelo transforma escores em uma distribuição de probabilidade e escolhe o próximo token. Reduzir a temperatura concentra a seleção em alternativas de maior escore, mas não estabelece um critério de verdade. Uma afirmação errada pode ser produzida de forma repetível. Da mesma maneira, exigir JSON pode facilitar a integração, porém não assegura que números, campos e justificativas estejam corretos.

O modelo local padrão é Llama 3.2 1B, selecionado por restrição de recursos da máquina, não por ter demonstrado maior qualidade pedagógica. O número de parâmetros, o formato de armazenamento, as ativações e o contexto contribuem para o consumo de memória. Para P parâmetros armazenados em b bits, $Pb/8$ fornece apenas uma aproximação dos bytes dos pesos; a memória efetiva inclui outras estruturas. Quantização reduz a precisão de armazenamento e pode diminuir esse custo, mas não elimina as demais parcelas.

A aplicação limita a inferência local a dois núcleos e solicita o descarregamento do modelo após a resposta. A troca reduz ocupação persistente de memória, ao custo de carregamento em solicitações futuras. Modelos por API transferem parte desse custo para o provedor, mas introduzem latência, indisponibilidade e limites de uso. Nenhum desses fatores é sinônimo de qualidade de raciocínio; por isso, conectividade, custo operacional e adequação pedagógica precisam ser avaliados separadamente.

2.4. Geração aumentada por recuperação

A geração aumentada por recuperação, ou RAG, combina geração com acesso a uma memória externa [Lewis et al. 2020]. Uma formulação geral considera documentos z recuperados e respostas condicionadas a eles:

$$p(y \mid x) \approx \sum_{z \in \mathrm{TopK}(x)} p_\eta(z \mid x)\, p_\theta(y \mid x, z). \tag{3}$$

Essa expressão descreve uma família de modelos; não é uma operação probabilística treinada no AutoTutor. O protótipo implementa uma cadeia de recuperação e composição de prompt: seleciona trechos, acrescenta sua origem ao contexto e solicita uma explicação. O gerador permanece fixo e o recuperador não é treinado conjuntamente.

A consequência prática é que importar documentos melhora a disponibilidade de evidências, sem garantir seu uso correto. Há três falhas distintas: a fonte adequada pode

não estar no acervo; pode existir, mas não ser recuperada; ou pode ser recuperada e interpretada incorretamente. O diagnóstico deve localizar a etapa, em vez de classificar qualquer resposta insatisfatória como defeito do modelo.

2.5. Representações lexicais, TF-IDF e BM25

A recuperação lexical representa textos por ocorrências de termos. Em TF-IDF, a frequência do termo no documento é ponderada por sua raridade no conjunto. O módulo de recomendação de problemas do protótipo utiliza espaços separados para enunciado, conceitos e resumo da abordagem. A similaridade do cosseno permite comparar a direção dos vetores, enquanto a combinação dos espaços dá maior peso à estrutura conceitual do que à narrativa. Esse mecanismo não deve ser confundido com embeddings neurais.

O acervo de estudo utiliza FTS5 do SQLite e BM25. Uma forma usual do escore é [Robertson and Zaragoza 2009]:

$$\mathrm{BM25}(q,d) = \sum_{t \in q} \mathrm{IDF}(t) \frac{f(t,d)(k_1+1)}{f(t,d) + k_1 \left(1 - b + b\frac{|d|}{|d|}\right)}. \tag{4}$$

A saturação da frequência reduz a vantagem de repetir um termo muitas vezes; a normalização de comprimento limita o efeito do tamanho do documento. Na implementação, títulos, tags e corpo recebem pesos diferentes. Normalização de acentos, equivalências português–inglês e bônus por correspondência de título complementam o ranqueamento. Para iniciantes, materiais identificados como fundamentos recebem preferência, sem excluir automaticamente outras fontes.

Esses critérios são controles explícitos de recuperação. Não atribuem domínio ao estudante nem geram uma aula a partir de condições fixas. Essa separação permite usar um método auditável para localizar evidências e reservar ao modelo a interpretação pedagógica do pedido. Um recuperador lexical também tem limites: duas descrições semanticamente próximas podem compartilhar poucas palavras.

2.6. Embeddings, similaridade semântica e fusão

Embeddings densos representam textos por vetores de dimensão fixa. Abordagens como Sentence-BERT permitem comparar representações de sentenças por similaridade, evitando avaliar cada par de textos por uma rede conjunta [Reimers and Gurevych 2019]. Para vetores u e v, utiliza-se

$$\mathrm{sim}(u,v) = \frac{u^\top v}{\|u\|\,\|v\|}. \tag{5}$$

A proximidade é uma medida aprendida de similaridade, não uma demonstração de equivalência algorítmica. “Consultar um intervalo” pode se referir a soma de prefixos, árvore de segmentos ou outra técnica, dependendo de haver atualizações.

Foi conduzido um experimento separado com `all-minilm:22m`, disponibilizado pelo Ollama como modelo de embeddings [Ollama 2026]. Ele não substitui o Llama na produção de texto. Foram representados títulos e os primeiros 1.200 caracteres dos 302 textos externos; portanto, o experimento não constitui um índice semântico de todos os fragmentos do acervo. A escolha de um modelo pequeno buscou limitar custo e memória, mas sua adequação ao português precisava ser medida.

A fusão de listas foi testada por *Reciprocal Rank Fusion* [Cormack et al. 2009]:

$$\mathrm{RRF}(d) = \sum_{r \in \mathcal{R}:\, d \in r} \frac{1}{k + \mathrm{pos}_r(d)}. \tag{6}$$

No experimento, $k = 60$ e as posições são indexadas a partir de zero. A técnica combina posições, sem exigir que escores lexicais e vetoriais estejam na mesma escala. Sua presença no experimento não significa que a busca semântica esteja ativada por padrão.

2.7. Personalização e limites da evidência educacional

Personalizar uma explicação exige distinguir preferências, desempenho observado e inferências sobre conhecimento. Uma leitura concluída é um evento de uso; um acerto é uma evidência pontual; repetidas tentativas no mesmo problema não equivalem a resolver vários objetivos diferentes. O sistema fornece ao modelo contagens de respostas, acertos, problemas distintos e problemas aceitos, além de vereditos recentes por assunto. Nomes, textos antigos e soluções de referência não são necessários nesse resumo.

O modelo recebe a instrução de utilizar somente estatísticas pertinentes para ajustar pré-requisitos, exemplos e exercícios. O pedido e o nível explicitamente escolhidos continuam prioritários. Assim, um histórico de DP não deve transformar um pedido de Segment Tree em outro exercício de DP. Não foi implementado um modelo probabilístico de domínio do aluno ou uma política de aprendizagem por reforço. A personalização atual é inferência condicionada a evidências resumidas; sua eficácia educacional ainda requer avaliação com estudantes.

3. Trabalhos relacionados e posicionamento

O AutoCode aborda a utilização de modelos de linguagem na construção de problemas e testes, enfatizando ciclos de geração e validação [Zhou et al. 2026]. O AutoTutor adota uma solução mais delimitada: o modelo escolhe entre contratos existentes e adapta sua apresentação, enquanto verificadores próprios preservam a semântica. Não se reivindica reprodução integral do método nem desempenho equivalente.

O OI-Assistant investiga recuperação para descoberta de problemas semelhantes e aprendizagem interativa [Su et al. 2025]. Essa perspectiva fundamenta a distinção entre proximidade narrativa e semelhança de estratégia. No protótipo, a busca de materiais e a recomendação de problemas constituem caminhos diferentes, com representações e critérios próprios. O Codigen explora autoria visual e modelos de linguagem em educação de programação [Ez-zaouia and Boumarafi 2025]. A aproximação do AutoTutor está na personalização e na apresentação didática, sem reproduzir o ambiente de autoria do trabalho original.

A contribuição desta implementação é articular essas motivações em um sistema local observável. O interesse científico não está em dar nomes de pesquisas a módulos, mas em explicitar quais mecanismos foram implementados, quais foram simplificados e quais hipóteses permanecem sem confirmação.

4. Arquitetura e metodologia

4.1. Fluxo de inferência e proveniência

A Figura 1 sintetiza o fluxo de materiais. O pedido escrito, ou o contexto transportado da página de estudo, determina a consulta. O recuperador retorna somente textos externos

elegíveis. Os metadados de vídeos, cursos e problemas servem para recomendações, mas não entram como se fossem conteúdo integral. O compositor inclui preferências, trechos e evidências resumidas do aluno. A resposta é salva com as fontes recuperadas e pode ser reaberta em uma página própria.

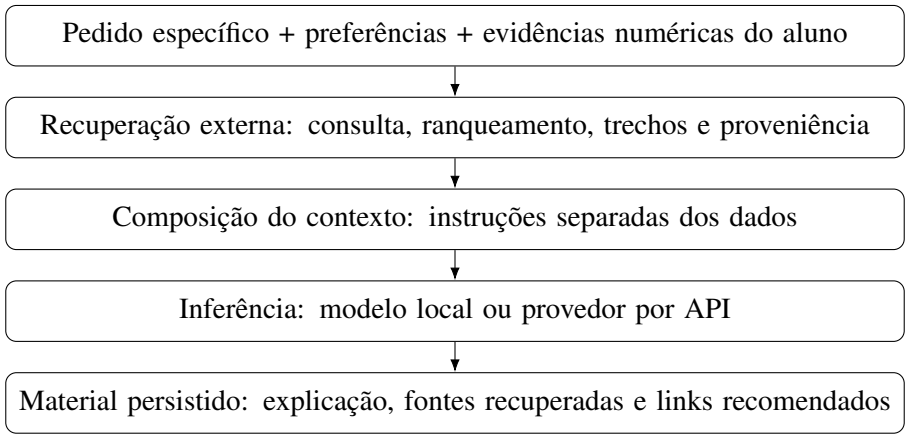


Figura 1. Fluxo implementado para geração de materiais. O armazenamento não atualiza os pesos do modelo.

Documentos externos podem conter instruções aparentes. Tratá-los como dados, e não como comandos de sistema, reduz a possibilidade de desviar a tarefa por *prompt injection*; entretanto, a delimitação textual não é uma prova de resistência a ataques. O sistema não executa o JavaScript baixado para extrair metadados. URLs, origem, licença, revisão e identificadores dos trechos permitem auditoria posterior.

A biblioteca preserva 19 textos autorais locais, mas os exclui por padrão do contexto externo de novos materiais. Essa decisão responde a um problema observado no protótipo: a aula interna podia aparecer repetidamente como principal evidência. Quando não há trecho externo recuperado, o material explicita que a explicação foi produzida a partir do conhecimento do modelo. Exibir um link no rodapé não comprova que sua página tenha sido lida.

4.2. Contratos de problemas e verificação independente

Um contrato define gramática de entrada, objetivo, limites, exemplos e critérios de saída. Existem 39 contratos locais: cinco de Segment Tree, seis de grafos e nove de DP, além de outros tópicos. O sistema prioriza contratos não usados recentemente. Se o assunto específico só possuir um contrato compatível, uma nova geração pode ser marcada como revisão; a aplicação não deve trocar o assunto apenas para aparentar diversidade.

Nos oito contratos acrescentados nesta etapa, soluções C++17 são confrontadas com oráculos Python. Para consultas em intervalos, a referência usa árvores de segmentos e o oráculo utiliza blocos e agregação independente. Em grafos, componentes e ciclos são conferidos por conjuntos disjuntos; bipartição é enumerada em casos pequenos; distâncias de múltiplas fontes são verificadas por caminhos mínimos em pequenos grafos. Casos maiores usam verificadores escaláveis. A independência é maior nos casos pequenos, mas não deve ser presumida como absoluta em todos os caminhos de código.

A aceitação em um conjunto finito X pode ser expressa por

$$\forall x \in X : \quad \text{valid}(x) \wedge f_{\text{C++}}(x) = f_{\text{oracle}}(x). \tag{7}$$

Isso fornece evidência empírica, não prova formal para todas as entradas possíveis. Exemplos publicados também são conferidos. Após a adaptação pela IA, a solução gerada é comparada à referência confiável, evitando que duas implementações concordem entre si após alterar indevidamente o objetivo. Compilação, execução e limites do juiz são mecanismos determinísticos; a seleção e a explicação são tarefas de IA. Misturar essas responsabilidades tornaria o diagnóstico menos confiável.

4.3. Corpus, atualização e persistência

O acervo reúne 302 textos externos: 164 do CP-Algorithms e 138 do USACO Guide, com respectivas licenças e revisões [CP-Algorithms contributors 2026, CP Initiative 2026]. Há também 5.230 referências externas deduplicadas, incluindo CSES, Neps, Maratona UFMG e YouKn0wWho. A contagem de referências não representa a quantidade de artigos integrais: vídeos não foram transcritos e cursos privados não foram copiados. O catálogo TLE recusou a consulta pública com HTTP 403; somente sua referência inicial está disponível.

Na atualização do USACO, 151 arquivos foram baixados com conferência de hash, e 138 tutoriais passaram pela limpeza de conteúdo. Uma repetição da importação classificou os mesmos 138 registros como inalterados. Corpos são comprimidos, enquanto o índice permite consulta textual. O banco de progresso permanece separado do corpus público, com consultas vinculadas ao perfil. Essa separação permite atualizar referências sem sobrescrever o histórico do aluno.

4.4. Desenho da avaliação exploratória

Foram adotados três níveis de avaliação. O primeiro verifica contratos e integração por testes automatizados. O segundo mede recuperação em 12 consultas em português: seis nomes explícitos de tópicos e seis paráfrases. O indicador *Hit@3* vale um quando ao menos um título esperado aparece entre os três primeiros resultados. O gabarito é definido por padrões de título, não por julgamento independente da relevância de cada trecho.

O terceiro nível executa cinco questões sintéticas no Llama 3.2 1B, abrangendo BFS, DFS, DP, prefixos e Segment Tree. Cada questão solicita um resultado numérico ou lista, justificativa e rótulos das fontes. A configuração usa temperatura zero, contexto de 2.048 tokens e limite de 256 tokens de saída. O critério automático exige o campo `result` com valor exato. Falta de campo, truncamento ou JSON inválido também produzem falha. Essa delimitação é necessária: o escore não isola competência matemática de aderência ao protocolo.

5. Resultados e discussão

Os oito novos contratos passaram pelos testes diferenciais e pela execução das soluções de referência. Existem pelo menos cinco objetivos distintos em cada um dos três tópicos priorizados. Os testes de roteamento verificam a manutenção do assunto, mas usam respostas controladas do provedor; não demonstram que o modelo local escolherá corretamente em qualquer pedido livre.

Na recuperação, a busca lexical obteve 6/12 acertos de tópico no *top-3*; a densa, 4/12; e a fusão RRF, 7/12. A matriz com 302 vetores em ponto flutuante de 32 bits ocupou 463.872 bytes, aproximadamente 453 KiB. Esse valor descreve somente os vetores, não os pesos do modelo ou a memória do processo. O experimento completo levou 21,28 segundos na execução registrada. Como a amostra é pequena e o índice denso representa apenas o início dos documentos, o ganho de um caso da fusão não justifica concluir superioridade geral. A busca lexical permaneceu como padrão.

Nenhuma das cinco respostas do modelo local satisfez o critério estrito de resultado e formato. A inspeção dos registros revela problemas além do formato: uma resposta confundiu profundidade com largura, outra atribuiu um único vértice alcançável a uma cadeia de quatro, e a resposta de Segment Tree apresentou números incompatíveis entre resultado e justificativa. No exemplo de prefixos, apareceu o valor esperado em uma chave não solicitada, acompanhado de uma justificativa incorreta. Portanto, seria inadequado interpretar o resultado simplesmente como "zero acertos matemáticos"; a conclusão sustentada é a ausência de respostas plenamente conformes e consistentes nesses cinco casos.

O experimento demonstra que recuperar uma fonte correta não força sua utilização correta. Restringir o formato também não substitui verificação semântica. Um modelo pequeno pode ser útil para explorar a interface e reduzir custo local, mas os dados observados não sustentam tratá-lo como tutor confiável para uso autônomo. Modelos de API não foram comparados nesta etapa, de modo que não há base experimental para eleger um provedor superior.

O fluxo de prática foi ajustado para transportar o pedido da página do estudo ou da busca até a geração do problema. Essa mudança preserva a intenção do usuário entre telas e permite registrar a origem da atividade. Ela resolve uma questão de contexto da aplicação; não elimina a possibilidade de o modelo falhar na seleção ou na implementação do contrato. Nesses casos, a atividade deve ser rejeitada ou sinalizada, em vez de publicada com outro assunto.

6. Limitações, validade e implicações

A avaliação não inclui participantes, retenção de conhecimento ou comparação com métodos tradicionais de ensino. Assim, melhoria de interface, aumento de contratos e aprovação nos testes não constituem evidência de ganho educacional. As consultas foram escolhidas pelos desenvolvedores e os padrões de título são uma aproximação de relevância. Não houve separação entre desenvolvimento do ranqueador e um conjunto amplo de teste independente.

O corpus combina idiomas e formatos, e a limpeza pode remover informação útil. Referências derivadas de URLs têm qualidade inferior à de metadados editoriais completos. Fontes recuperadas são rastreáveis, mas sua presença não assegura que cada afirmação gerada seja sustentada por elas. O orçamento curto do modelo pode influenciar o teste de formato; ainda assim, inconsistências numéricas e conceituais observadas merecem avaliação adicional.

Também há limites no juiz: concordância em testes finitos não garante correção, e execução local com limites de recursos não deve ser apresentada como isolamento suficiente para receber código de usuários não confiáveis na internet. Uma implantação pública

exige avaliação específica de isolamento e operação. No âmbito de IA, mantém-se a distinção entre filtros operacionais, evidência do aluno e decisão pedagógica: controles determinísticos são necessários para integridade, mas não substituem a análise contextual realizada pelo modelo.

7. Conclusão e trabalhos futuros

O AutoTutor materializa uma integração de IA generativa, recuperação externa, personalização por evidências e verificação executável. O estudo mostrou a importância de distinguir quatro afirmações: há uma fonte disponível; a fonte foi recuperada; a resposta a utilizou corretamente; e o problema gerado foi validado. Nenhuma dessas etapas implica automaticamente a seguinte.

Os resultados justificam manter proveniência explícita, impedir que referências sem texto sejam usadas como evidência e tratar o modelo local como recurso experimental. Como próximos trabalhos, propõem-se avaliação humana de fundamentação, conjunto independente de consultas, embeddings multilíngues e por fragmento, comparação controlada de geradores e estudo com alunos. A elaboração de mais contratos deve continuar acompanhada por oráculos e testes, preservando a possibilidade de praticar o assunto atual sem transformar uma mudança de narrativa em suposta diversidade algorítmica.

Referências

Cormack, G. V., Clarke, C. L. A., and Büttcher, S. (2009). Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of SIGIR*, pages 758–759. DOI: 10.1145/1571941.1572114.

CP-Algorithms contributors (2026). Algorithms for competitive programming. `https://cp-algorithms.com/`. Revisão bbbbb6f04e5c9a665c8477ec68ee4f28b1367dd7. Licença CC BY-SA 4.0.

CP Initiative (2026). Usaco guide. `https://usaco.guide/`. Repositório cpinitiative/usaco-guide, revisão a2a25966a3c8e6cb7e0a87985b0abaac99e2ad40. Licença CC BY-NC-SA 4.0.

Ez-zaouia, M. and Boumarafi, Y. (2025). Codigen: Enhancing personalized programming education with visual authoring and llms. In *IEEE Symposium on Visual Languages and Human-Centric Computing*. DOI: 10.1109/VL-HCC65237.2025.00038.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Kuttler, H., Lewis, M., Yih, W.-t., Rocktaschel, T., Riedel, S., and Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474. arXiv:2005.11401.

Ollama (2026). Generate embeddings; all-minilm model library. `https://docs.ollama.com/api/embed`. Modelo: `https://ollama.com/library/all-minilm`. Acesso em novembro de 2026.

Reimers, N. and Gurevych, I. (2019). Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of EMNLP-IJCNLP*, pages 3982–3992. DOI: 10.18653/v1/D19-1410.

Robertson, S. and Zaragoza, H. (2009). The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389. DOI: 10.1561/1500000019.

Su, Y., Nie, P., and Meng, X. (2025). Oi-assistant: A retrieval augmented system for similar problem discovery and interactive learning in competitive programming. *Olympiads in Informatics*, 19:129–144. DOI: 10.15388/ioi.2025.09.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30. arXiv:1706.03762.

Zhou, S., Zheng, Z., Liu, K., Shen, Z., Cheng, Z., et al. (2026). Autocode: Llms as problem setters for competitive programming. In *International Conference on Learning Representations*. arXiv:2510.12803.